

רמזים

שים לב

שלוש דרגות בכל קושי. קרא את הדחיפה, נסה עשר דקות, ורק אז את האסטרטגיה. הכמעט-פתרון הוא הרשת, לא הקיצור. שישה עשר קשיים, מסודרים לפי הסדר שבו הם קורים — קודם האחסון (חלקים א-ב), אחר כך הרשת (ג-ז). **שלושה כלים פותרים את רובם:** DevTools > Application > Local Storage (מה באמת נשמר), Network מסונן ל-Fetch/XHR (אם בקשה יצאה, מה הסטטוס, מה חזר), ו-Elements על #api-panel (ה- data-state אומר מה היישום חושב שקורה).

א · "שמרתי, רעננתי, וכלום לא חזר"

דחיפה. פתח את Application > Local Storage. **יש שם מפתח?** התשובה מפצלת את הבעיה לשתיים: כתיבה או קריאה. **אסטרטגיה.** אם אין מפתח — `persist` לא כותב: או ש-`save` עוד מחזירה בלי לכתוב, או שהשומר הראשון יוצא תמיד (הוא משווה למשהו שכבר שווה). אם יש מפתח — `load` לא קורא אותו, או קורא מפתח בשם אחר ממה שנכתב. **כמעט-פתרון.** KEY אחד, בראש הקובץ, ושתי הפונקציות משתמשות בו — לא מחרוזת שמוקלדת פעמיים. ובדוק שהמטפלים בונים מערכים חדשים (`[...items, item]`, `filter`, `map`): `push` משנה את המערך הקיים, השומר של `persist` רואה "אותו מערך", ושום דבר לא נכתב.

ב · "העמוד ריק, ובקונסול SyntaxError או map is not a function."

דחיפה. מה יושב במפתח עכשיו? תסתכל ב-Application. ותריץ עליו `JSON.parse` בקונסול, ידנית. **אסטרטגיה.** אם `load` זורקת, היא זורקת בשורה הראשונה של `app.js` — לפני שמישהו נרשם ולפני שמישהו צויר. `SyntaxError` הוא `parse` שלא עטוף. `map is not a function` הוא `parse` שהצליח על משהו שאינו מערך — `null`, או `{"not": "an array"}` — ומגיע ל-`hydrate`. **כמעט-פתרון.** שלושה שלבים, בסדר הזה: `try` סביב `getItem`; `raw === null`; `[]` בלי משפט; `try` סביב `JSON.parse`; ואז `Array.isArray` — ורק אז `filter` של השורות. זה מחזור 2, צעד 2.

ג · "שמתי try/catch וזה עדיין נופל"

דחיפה. שים את המחרוזת שהפילה אותך בקונסול ותריץ עליה `JSON.parse`. אם היא לא זרקה — ה-`try` שלך לא קשור לבעיה. **אסטרטגיה.** `JSON.parse` שהצליח אומר שהמחרוזת תקינה, ולא כלום על המבנה. שלושה מתוך ארבעת הערכים שהבודק שם במפתח עוברים את ה-`parse` בשלום: `null`, `{"not": "an array"}`, ומערך עם שורה אחת מקולקלת. **כמעט-פתרון.** אחרי ה-`parse`: זה מערך? ואז כל שורה — אובייקט, עם מזהה שהוא מחרוזת לא ריקה, שם, מספר שהוא באמת מספר, ומדף. רק השדות שהיישום באמת קורא. שורה שנכשלת נושרת; השאר נשארות.

ד · "מחקתי את המפתח ורעננתי — וקיבלתי את נתוני הדוגמה"

דחיפה. מאיפה הם הגיעו? חפש את `SEED_ITEMS` בקוד שלך. כמה מקומות?

אסטרטגיה. בחוזה של הפרויקט **מפתח שאינו קיים הוא מזוהה ריק**. נתוני הדוגמה מגיעים רק מהכפתור "טען פריטים לדוגמה" (`events.js`, נתון). אם `load` מחזירה אותם כשאין מפתח — או ש-`app.js` זורע כשהמערך ריק — ביקור ראשון נראה "מלא", ובשבוע 13 זה מה שמסתיר את התקלה של גרסה ב.

כמעט-פתרון. `load` מחזירה `[]` כשאין מפתח, ו-`app.js` נשאר `hydrate(load())` — כמו שקיבלת אותו. אל תיגע בו.

ה · "הדף קופא, ובקונסול Maximum call stack size exceeded"

דחיפה. מה `persist` עושה אחרי שהיא שומרת? ומי קורא ל-`persist`?

אסטרטגיה. `persist` רשומה על `setState`. אם `persist` קוראת ל-`setState` — `setState` מודיעה לכל המנויים, כולל ל-`persist`, שקוראת ל-`setState` ... עד שהמחסנית נגמרת. זה מחזור 1, צעד 3.

כמעט-פתרון. `setState` בתוך `persist` רק כשהמשפט השתנה: השווה את המשפט שחזר מ-`save` למה שכבר ב-`state.storageNote`. בסיבוב השני הם שווים, ושום דבר לא קורה.

ו · "הערך ב-Application לא נראה כמו רשימה"

דחיפה. במה מתחיל הערך — ב-`[]` או ב-`{}`?

אסטרטגיה. הערך הוא המערך עצמו: `JSON.stringify(items)`. לא `{ items }`, לא `{ v: 1, items }`, ולא ה-`state` כולו — חיפוש, מיון ושורה שנערכת הם לא נתונים של המשתמש. והמפתח: שם, נקודתיים, `v` ומספר — `pantry:v1` כבר כתוב לך בשלד.

כמעט-פתרון. `save(items)` מקבלת את המערך, ו-`persist` שולחת לה `state.items`.

ז · "הטופס לא מתמלא כשאני לוחץ 'ערוך'"

דחיפה. המטפל של `button.edit` בכלל רץ? שים `console.log` בתוכו.

אסטרטגיה. המאזין הוא אחד, על `#list`, ובודק לאיזה כפתור הלחיצה שייכת עם `closest`. הלחיצה נוחתת לרוב על ה-`<span>` שבתוך הכפתור, לא על הכפתור.

כמעט-פתרון. בענף העריכה: מצא את הפריט לפי המזהה של השורה, מלא את הטופס (`fillForm` כבר כתובה), ו-`setState({ editingId: id })`. שום דבר עוד לא נערך — רק ההצעה לערוך.

ח · "העריכה מוסיפה שורה חדשה במקום לשנות"

דחיפה. בשליחה, מה מחליט אם זו הוספה או עדכון?

אסטרטגיה. `selectEditing(state)` — אם הוא לא `null`, זו החלפה: אותו מזהה, אותה כמות שורות.

כמעט-פתרון.

`items.map((item) => (item.id === editing.id ? { ...item, ...fields } : item))`, ואז `editingId: null` באותו `setState`. אל תשנה את הפריט שמצאת (`found.title = ...`) — זה עובד על המסך ושובר את כל מה שבא אחריו.

ט · "לא קורה כלום כשאני מקליד בקטלוג"

דחיפה. Network, מסונן ל-Fetch/XHR. הקלד. **יצאה בקשה?**

אסטרטגיה. אם לא — הבעיה לפני הרשת: המאזין על `#api-query`, `scheduleSearch` שיוצאת מוקדם, או טיימר שנדרס. אם כן — הבעיה אחריה: הסטטוס, ולשונית `Response`.

כמעט-פתרון. `Uncaught (in promise)` בקונסול — הפונקציה שלך `async`, היא זרקה, ואיש לא תפס: `runSearch` נקראת מטיימר, ואף אחד לא ממתין לה.

י · "422 או 404 מציגים לי 'לא נמצא כלום' במקום שגיאה"

דחיפה. `fetch` לא זרק. הוא לא זרק על 404, על 422 ועל 500. מה `res.ok` מחזיר?

אסטרטגיה. ההבטחה של `fetch` נדחית רק כשהבקשה לא הושלמה — DNS, אין נתיב, CORS, ביטול. סטטוס הוא תשובה: השרת ענה "לא", ולפעמים בגוף JSON תקין לחלוטין — ואז המגרמל שלך מוצא בו "אפס תוצאות". זה מחזור 3, צעד 1.

כמעט-פתרון. שורה אחת, מיד אחרי ה-`await fetch` ולפני ה-`json()`:

```
if (!res.ok) throw new ApiError('status', `... (${res.status})`, `HTTP ${res.status}`);
```

יא · "התשובה חוזרת 200, ואני מקבל שגיאה"

דחיפה. לשונית Network < הבקשה < `Response`. מה חזר? הרץ עליו `JSON.parse` בקונסול.

אסטרטגיה. שלוש אפשרויות, ובכולן `res.ok` הוא `true`: הגוף אינו JSON (עמוד התחברות של Wi-Fi), הגוף JSON במבנה אחר, או מגרמל שמניח מפתח שלא קיים.

כמעט-פתרון. `payload.results.map(...)` על `{}` הוא `TypeError` שנזרק מתוך ה-`try` של הרשת ונראה כמו תקלת רשת. הרבה APIs משמיטים את מפתח התוצאות כשלא נמצא כלום — בדוק את שלך בשורת הכתובת.

יב · "ספינר ולא נמצא כלום' מוצגים ביחד"

דחיפה. כמה שדות ב-state מחליטים איזה אזור מוצג? אם יותר מאחד — מצאת.

אסטרטגיה. בוליאני `loading` לצד `status` הם שני שדות שצריכים להשתנות יחד ואפשר לשנות לחוד (מחזור 4, צעד 1). `loading` הוא ערך של השדה, ו-`empty` נגזר מהתוצאות.

כמעט-פתרון. `render` מחשב את התצוגה פעם אחת (`selectSearchView`), כותב אותה ל-`data-state`, וכל אזור מוצג רק כשהוא שווה לה. ובדוק ש-`status: 'loading'` נכתב לפני ה-`await`.

יג · "'לא נמצא כלום' כתוב כבר בטעינת העמוד"

דחיפה. בטעינה `results` ריק. מה הגזירה שלך מחזירה עבורו?

אסטרטגיה. `idle` ו-`empty` הם שני מצבים: אחד שולח לחיבת החיפוש, השני אומר שהיא כבר אכזבה.

כמעט-פתרון. בדיקת `'idle' === status` באה לפני בדיקת האורך.

יד · "TypeError: Failed to fetch" ואין שום פרט

דחיפה. הכתובת עובדת בשורת הכתובת? ובקונסול — מה **הדפדפן** כתב, בשורה נפרדת?

אסטרטגיה. אם כתוב "blocked by CORS policy" — הבקשה יצאה, השרת ענה, והדפדפן סירב למסור לך את התשובה, כי לא הייתה בה הכותרת `access-control-allow-origin`. הסיבה מודפסת בקונסול ואין דרך לקרוא אותה מ-JavaScript. ואם אין שורה כזאת — זו הרשת.

כמעט-פתרון. אין תיקון בקוד שלך. `mode: 'no-cors'` נותן תשובה **אטומה** (סטטוס 0, בלי גוף). **בחר API ששולח את הכותרת** — ולפעמים זה פרמטר: ה-API של ויקיפדיה שולח אותה רק עם `origin=*`. תעד בהערה את הבקשה שנחסמה (חלק ג.1).

טו · "fetch לקטלוג המקומי נכשל", או "שום דבר לא עובד"

דחיפה. מה כתוב בשורת הכתובת? אם `file:///` — זה זה.

אסטרטגיה. מ-`file://` אין `origin`, והמודולים עצמם לא נטענים:

— `fetch` — `Access to script ... from origin 'null' has been blocked by CORS policy`. לא רק ה-`fetch` — שום דבר.

כמעט-פתרון. Live Server, תמיד. ואם הוא רץ ועדיין 404 — `fetch('data/catalogue.json')` יחסי ל-**עמוד**, לא לקובץ ה-JS.

טז · "הפריט מהקטלוג לא נשמר", או "ההצעה לא חוזרת כשמוחקים"

דחיפה. Application: הפריט בפנים? אם כן — הבעיה בקריאה; אם לא — בכתיבה.

אסטרטגיה. אתה **לא כותב** קוד שמירה בחלק ה. המטפל קורא ל-`setState`, ו-`persist` רשומה עליה. אם זה לא נשמר — המטפל לא עבר דרך `setState` עם מערך **חדש**. ואם ההצעה לא חוזרת אחרי מחיקה — מחקת את השורה ביד במקום לגזור.

כמעט-פתרון. `setState({ items: [...state.items, newItem] })`, ורשימת ההצעות נגזרת: ההצעות שהכותרת שלהן לא נמצאת באוסף.

כשהרשת של המעבדה נופלת

`node mock-api/server.mjs` (התיקיה `mock-api/` בחומרי השבוע), ומכוונים את `ENDPOINT` אליו. יש שם `README-he.md` קרא את `?hang=1` — כל התקלות, בפקודה. קרא את `?fail=500`, `?fail=404`, `?delay=8000` — שלידו.

שלוש בדיקות של שלושים שניות, לפני שאתה מגיש

1. מחק את המפתח ב-Application ורענן: **מזווה ריק**, עם המשפט.

2. שים `{"not": "an array"}` במפתח ורענן: העמוד עולה, עם משפט.

3. כבה את ה-Wi-Fi והקלד בקטלוג: משפט, והקטלוג המקומי עובד.